

✓ What is Lifecycle in Flutter?

In Flutter, **lifecycle** means the stages a widget goes through from the time it is created until it is removed from the screen.

- 👉 Just like humans have life stages (birth → growth → death),
- 👉 Widgets also have stages in their “life”.

Real-Life Analogy (Easy to Teach) 🌱 Plant

Lifecycle

Seed → Sprout → Plant → Flower → Wither

Flutter Widget:

Create → Build → Update → Dispose

🎯 Why Lifecycle is Important?

Because it helps you:

- Load data when screen starts
 - Update UI when data changes
 - Free memory when screen closes
 - Improve performance
-

Lifecycle in StatefulWidget (Most Important)

Lifecycle is mainly discussed for **StatefulWidget**.



Main Lifecycle Methods

1 createState()

- 👉 Called when widget is created
 - 👉 Creates mutable state
-

2 initState()

- 👉 Called once when screen loads
- 👉 Best place to:
 - Fetch API data
 - Initialize variables
 - Start animations

Example:

```
@override
void initState() {
  super.initState();
  print("Screen started");
}
```

3 build()

- 👉 Called many times
- 👉 Builds the UI
- 👉 Runs whenever UI updates

```
@override
Widget build(BuildContext context) {
  return Text("Hello");
}
```

4 **setState()**

- 👉 Not exactly a lifecycle method but triggers rebuild
- 👉 Updates UI

```
setState(() {  
  count++;  
});
```

5 **dispose()**

- 👉 Called when widget is removed
- 👉 Used to:

- Close streams
- Stop timers
- Free resources

```
@override  
void dispose() {  
  print("Screen closed");  
  super.dispose();  
}
```



Simple Lifecycle Flow

```
createState()  
↓  
initState()  
↓  
build()  
↓  
setState() → build() again  
↓  
dispose()
```

Example->

Write:

```
initState() {  
  print("Start");  
}  
  
build() {  
  print("Building");  
}  
  
dispose() {  
  print("End");  
}
```

Diagram

```
graph TD  
  Start --> initState  
  initState --> build  
  build --> dispose
```

One-Line Explanation

"Lifecycle is the life journey of a widget from appearing to disappearing on screen."

Stateless vs Stateful Widgets in Flutter

Flutter has two main types of widgets:

- 1 Stateless Widget
- 2 Stateful Widget

They decide whether your UI can change or not.

◆ 1 Stateless Widget

📦 Simple Meaning

A Stateless widget is a widget whose data **cannot change** after it is built.

👉 UI remains fixed.

🧠 Real-Life Analogy

📄 Printed Photo

- Once printed, it cannot change
 - Same like Stateless widget
-

🔧 Example

```
class MyText extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text("Hello");  
  }  
}
```

👉 Text will always stay "Hello"

📌 Use When

- Showing text
 - Icons
 - Static UI
 - Labels
 - Images
-

◆ 2 Stateful Widget

📦 Simple Meaning

A Stateful widget is a widget whose data **can change** during runtime.

👉 UI updates when data changes.

🧠 Real-Life Analogy

📱 Instagram Likes

- Likes increase/decrease
 - Screen updates automatically
 - That's Stateful
-

🔧 Example

```
class Counter extends StatefulWidget {  
  @override  
  _CounterState createState() => _CounterState();  
}
```

```
class _CounterState extends State<Counter> {  
  int count = 0;
```

```
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Text("$count"),  
        ElevatedButton(  
          onPressed: () {
```

```

        setState(() {
          count++;
        });
      },
      child: Text("Increase"),
    )
  ],
);
}
}

```

👉 Number changes when button pressed

Difference Table

Feature	Stateless	Stateful
Data change	❌ No	✅ Yes
UI update	❌ No	✅ Yes
setState()	❌ Not used	✅ Used
Example	Text, Icon	Counter, Form

Simple Rule to Remember

No change → Stateless

Changes → Stateful

Fun Classroom Activity

Ask students:

👉 Is this Stateless or Stateful?

- Clock 🕒 → ??
 - Profile Picture → ??
 - Score Counter → ??
 - Logo → ??
-

One-Line Explanation

"Stateless widgets don't change. Stateful widgets can change and update the screen."